哈爾濱工程大學

第 15 讲 / 全 17 讲

C 语言结构体用法详解

把不同类型的数据捆成一个整体，一个学生就是一个变量

自定义类型

自己造一个类型，把不同类型捆在一起

点运算符

s.name 、 s.age ：用点把成员取出来

结构体数组

30 个学生放进一个数组里一起管

结构体指针

传地址才改得动， p->age 找成员

本节目标

六条，学完逐条自查

哈尔滨工程大学

1. 说出为什么需要结构体：一个人的信息不是一个数
2. 用 struct 定义自己的类型，记住结尾那个分号不能忘
3. 用点运算符读写成员： s.name 、 s.age 、 s.score
4. 定义结构体数组，把 30 个学生放进一个数组里遍历
5. 会用 typedef 起别名，会用箭头 -> 通过指针访问成员
6. 把结构体传进函数，并能按成绩给结构体数组排序

01

结构体基础

为什么需要它 · 怎么定义类型 · 怎么声明变量 · 怎么访问成员

为什么需要结构体

```
// 一个学生，被拆成了四个变量
char   name1[20];    // 姓名
int    id1;          // 学号
int    age1;         // 年龄
double score1;       // 成绩
char   name2[20];    // 第二个学生
int    id2;
int    age2;
double score2;
// 100 个学生 = 400 行声明
// 排序时成绩换了，名字没动
```

一个人被拆成四份

name1 、 id1 、 age1 、 score1 谁也不认识谁，看代码才知道它们属于同一个人

人多就疯狂复制

再来一个学生就得再抄四行， 100 个人要写 400 行，改一处要改一百遍

排序会串味

成绩数组排好了序，姓名数组还留在原地，第一名配上了别人的名字

结构体是自定义类型

int 装一个数，double 装一个小数，char 数组装一串字符——都是现成类型。

但一个学生要同时装姓名、学号、年龄、成绩，C 语言没有现成的类型可用。

struct 让你自己造一个：把不同类型捆在一起，取个名字再用。

自带类型

int

装一个整数

18

自带类型

double

装一个小数

92.5

自带类型

char[20]

装一串字符

张三

自己造的类型

Student

一次装完一整个学生

姓名 + 学号 + 年龄 + 成

造好以后，Student 就和 int 一样：能定义变量、能开数组、能用指针指向它。

定义 struct Student

```
struct Student {
    char name[20];    //  姓名
    int id;           //  学号
    int age;          //  年龄
    double score;      //  成绩
};
//  分号最容易漏，漏了下一行报错
//  定义类型只是一次声明，不占内存
//  拿它去定义变量，才真的有空间
```

类型名是两个词

struct Student 合起来才是类型名

花括号是成员清单

每行一个成员，各有自己的类型

分号千万别漏

}; 后面没分号，编译器立刻翻脸

定义不占内存

要声明变量才真的分到空间

定义放在哪里都行

三种位置，推荐第一种

哈尔滨工程大学

① 写在 main 外面

```
// 全局，谁都能用
struct Student {
    char name[20];
    int id;
};
int main(void) {
}
```

推荐：类型定义放最前面，所有函数都能用它声明变量

② 写在函数内部

```
int main(void) {
    // 只在本函数内
    struct Student {
        char nm[20];
        int id;
    };
}
```

出了这个函数就失效，别的函数认不出这个类型

③ 顺手声明变量

```
// 定义完立刻声明
struct Student {
    char name[20];
    int id;
} s1, s2;
int main(void) {
}
```

类型和变量混在一起，读起来乱，实际很少这么写

三种写法语法上都没错，区别只在 " 这个类型谁能看见 " 。

课上和作业一律用 ①：结构体定义放在 main 前面，清清楚楚。

声明变量与初始化

```
// ① 先声明，成员值还没定
struct Student s1;
// ② 声明时一次性给全
struct Student s2 = {"张三", 1001, 18, 92.5};
struct Student s3 = {"李四", 1002, 19};  // 后面补 0
struct Student s4 = {0};          // 全部清零
// ③ 字符数组成员要用 strcpy
strcpy(s1.name, "王五");
s1.id = 1003;
s1.age = 18;
s1.score = 88.0;
```

花括号按序给值

值的顺序必须和成员定义的顺序一致，不能跳着给

个数不够就补 0

只写前三个，剩下成员自动填 0；写 {0} 表示全清零

字符串成员用 strcpy

数组不能直接 s1.name = "王五"，要用 strcpy 拷贝

成员访问：点运算符

```
// 有了变量，才能访问它的成员
struct Student s = {"张三", 1001, 18, 92.5};
printf("%s", s.name);          // 张三
printf("%d", s.id);            // 1001
printf("%d", s.age);           // 18
printf("%.1f", s.score);       // 92.5
// 成员就是普通变量，能读也能写
s.age = s.age + 1;
s.score = 95.0;
printf("%s %d", s.name, s.age);  // 张三 19
```

点就是 " 的 "

s.age 读作 "s 的 age" ，一个点把成员取出来

成员是普通变量

能读能写，能放进表达式参与运算，和 int 变量没区别

格式符看成员类型

%s 打字符串、 %d 打整数、 %.1f 打小数

02

输入输出与数组

scanf 读成员 · 结构体数组 · 嵌套 · typedef 别名 · 整体赋值

scanf 读结构体成员

```
struct Student s;
//  键盘上敲：张三  1001 18 92.5
scanf("%s", s.name);     //  数组名不加  &
scanf("%d", &s.id);      //  取成员地址
scanf("%d", &s.age);     //  取成员地址
scanf("%lf", &s.score);  //  小数用  %lf
printf("%s %d", s.name, s.id);
printf(" %d %.1f", s.age, s.score);
//  屏幕：张三  1001 18 92.5
```

取成员的地址

&s.id 取的是 s 里 id 的地址

呼应 scanf 课

scanf 要的是地址，普通成员必须加

数组名不加符号

name 是数组名，本身就是地址

小数用 %lf

score 是 double ，写成 %f 读出垃圾

结构体数组：一整班学生

struct Student a[30]; 一句话开出 30 个学生的空间，a[0] 到 a[29]。

每个格子里都完整地装着姓名、学号、年龄、成绩，中间 a[4] 到 a[28] 结构完全一样。

a[0]	a[1]	a[2]	a[3]	a[29]
name[20] 张三	**name[20]** 李四	**name[20]** 王五	**name[20]** 赵六	**name[20]** 郑七
id 1001	**id** 1002	**id** 1003	**id** 1004	**id** 1030
age 18	**age** 19	**age** 18	**age** 19	**age** 18
score 92.5	**score** 78.0	**score** 85.5	**score** 61.0	**score** 70.0

结构体数组的遍历


```
// 声明时一次性给三个学生
struct Student a[3] = {
    {"张三", 1001, 18, 92.5},
    {"李四", 1002, 19, 78.0},
    {"王五", 1003, 18, 85.5}
};
// 用 for 一个一个走过去
int i;
for (i = 0; i < 3; i++) {
    printf("%s %d", a[i].name, a[i].id);
    printf(" %.1f", a[i].score);
}
```

a[i] 是第 i 个学生

a[0] 是张三，a[1] 是李四，一次取到一整个学生

先取元素再取成员

a[i].score 读作 " 第 i 个学生的成绩 "

下标从 0 开始

和数组课一模一样，i < 3 别写成 i <= 3

结构体嵌套：学生有生日


```c
// 先定义内层：生日
struct Date {
    int year, month, day;
};
// 外层：生日当一个成员塞进去
struct Student {
    char name[20];
    struct Date birthday;
    double score;
};
// 内层 {2006, 3, 15} 整块给 birthday
struct Student s = {"张三", {2006, 3, 15}, 92.5};
printf("生日：%d-%d-%d", s.birthday.year,
       s.birthday.month, s.birthday.day);
```

成员可以是结构体

birthday 的类型是 struct Date ，它自己又有三个成员

两层点往里走

s.birthday.year 读作 "s 的生日的年 "

内层也要花括号

初始化时 {2006, 3, 15} 整块给 birthday

typedef 起个短名

```
// typedef 旧名字 新名字
typedef struct Student {
    char name[20];
    int id;
    int age;
    double score;
} Student;   // 别名在这
Student s1;         // 不用写 struct
Student a[30];      // 一整班
Student *p = &s1;   // 指针也行
// 类型名从两个词缩成一个词
```

旧名换小名

给 struct Student 起了个小名叫 Student

以后少写 struct

Student s1; 和 struct Student s1; 一样

别名写在分号前

} Student; 顺序写反就编译不过

别名不是变量

typedef 只是个名字，不占任何内存

整体赋值：整份拷贝

```
struct Student s1 = {"张三", 1001, 18, 92.5};
struct Student s2;
s2 = s1;                  //  整份拷贝
// s2  现在是张三的复制品
printf("%s", s2.name);          //  张三
printf("%d", s2.id);            // 1001
printf("%.1f", s2.score);       // 92.5
//  数组元素也能整份赋值
struct Student a[3];
a[0] = s1;              //  搬进数组
```

同类型才能等号

两边都是 struct Student 才能整份赋值

等于逐个复制

所有成员一起过去，不用一个一个写

数组元素也能搬

a[0] = s1; 把整份数据放进格子

排序就靠这一句

交换时名字跟着成绩一起走，不会串味

03

结构体与函数

传值 · 传指针 · 箭头运算符 · 结构体返回值 · 排序一整班

哈爾濱工程大學

结构体作函数参数

```c
// 形参 s 是实参的一份完整拷贝
void show(struct Student s) {
    printf("%s", s.name);
    printf(" %d", s.id);
    printf(" %.1f", s.score);
}
int main(void) {
    struct Student s1 = {"张三", 1001, 18, 92.5};
    show(s1);      // 整份拷过去
    return 0;
}
// 屏幕：张三 1001 92.5
```

传值 = 整份拷贝

实参每个成员都复制一份给形参，两边是两块内存

改不动实参

呼应函数课值传递：函数里改 s.age ，外面看不见

好处与代价

安全不会误改，但成员多时拷贝花时间

指针传参与箭头运算符

```c
// 形参改成指针，才能改到实参
void raise(struct Student *p) {
    p->age = p->age + 1;
    p->score = p->score + 5;
}
int main(void) {
    struct Student s1 = {"张三", 1001, 18, 92.5};
    raise(&s1);   // 交出地址
    printf("%d", s1.age);     // 19
    printf(" %.1f", s1.score); // 97.5
    return 0;
}
```

想改就传地址

呼应指针课：把 &s1 交出去，函数才找得到本人

箭头怎么读

p->age ： p 指向的那个结构体的 age

只拷一个地址

不管结构体多大，传指针的代价都一样小

点与箭头：什么时候用哪个

哈尔滨工程大学

手里是结构体变量 → 用点

```
struct Student s;
s.age = 19;
printf("%d", s.age);
printf("%s", s.name);
```

s 本身就是一整块结构体，点一下就把成员取出来了。

管它多大，s.age 永远是 "s 的 age"。

手里是指针 → 用箭头

```
struct Student *p = &s;
p->age = 19;
printf("%d", p->age);
printf("%s", p->name);
```

p 只是一张写着地址的纸条，得先顺着地址找到人。

箭头就是 " 先去找，再取成员 " 。

(*p).age 和 p->age 完全等价，箭头只是省事的简写，看着更像 " 指过去 " 。

口诀：变量用点，指针用箭头。形参写成 struct Student *p ，里面就一律用 -> 。

结构体作返回值

```
//  返回类型写  struct Student
struct Student make(int id) {
    struct Student t;
    strcpy(t.name, "新生");
    t.id = id;
    t.age = 18;
    t.score = 60.0;
    return t;              //  整份返回
}
int main(void) {
    struct Student s1 = make(1001);
    printf("%s %d", s1.name, s1.id);
    return 0;
}
```

返回类型写在最前

函数头第一行就写清楚要交出 struct Student

return 整份拷贝

局部变量 t 的所有成员一起拷回去，函数结束也不丢

用同类型变量接住

左边必须是同类型： s1 = make(1001);

给结构体数组排序

呼应排序课：交换整个结构体

哈爾濱工程大學 1953 HARBIN ENGINEERING UNIVERSITY

```c
//  冒泡排序，按成绩从高到低
void sort(struct Student a[], int n) {
    int i, j;
    struct Student tmp;
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - 1 - i; j++)
            if (a[j].score < a[j + 1].score) {
                tmp = a[j];  //  整份搬走
                a[j] = a[j + 1];
                a[j + 1] = tmp;
            }
}
```

比较只看成员

a[j].score < a[j + 1].score 只比成绩

交换整个结构体

tmp = a[j] 一搬就是一整个学生

名字不会串味

姓名学号跟着成绩一起走

外壳和排序课一样

两层 for 和排序课一样

结构体设计小结

① 定义类型

```
struct Student { ... };
```

struct + 名字 + 成员清单 + 结尾分号，放在 main 前面

② 访问成员

```
s.age       p->age
```

变量用点，指针用箭头；(*p).age 和 p->age 一样

③ 组织数据

```
Student a[30];
```

结构体数组装一整班， typedef 起个短名少写很多字

④ 传进函数

```
void f(Student *p)
```

只想读就传值，想改实参就传指针，排序用整体赋值

一句话：结构体把散装变量打包成一个整体，从此一个学生就是一个变量。

例 1 · 录入并打印学生

```c
struct Student {
    char name[20]; int id; double score;
};
int main(void) {
    struct Student a[3]; int i;
    for (i = 0; i < 3; i++)
        scanf("%s%d%lf", a[i].name, &a[i].id,
              &a[i].score);
    for (i = 0; i < 3; i++)
        printf("%s %d %.1f", a[i].name,
               a[i].id, a[i].score);
    return 0;
}
```

题目

从键盘读入 3 个学生的姓名、学号、成绩，再原样打印出来

运行结果

```
键盘：张三 1001 92.5
键盘：李四 1002 78.0
屏幕：张三 1001 92.5
屏幕：李四 1002 78.0
```

要点

数组元素 a[i] 是结构体，成员前加点

name 是数组名不加 & ， id 和 score 要加

例 2 · 找最高分学生

```c
struct Student { char name[20]; double score; };
int main(void) {
    struct Student a[3]; int i, best = 0;
    for (i = 0; i < 3; i++)
        scanf("%s%lf", a[i].name, &a[i].score);
    // 打擂台：先假设 a[0] 最高
    for (i = 1; i < 3; i++)
        if (a[i].score > a[best].score)
            best = i;   // 只记下标
    printf("%s %.1f", a[best].name,
           a[best].score);
    return 0;
}
```

题目

读入 3 个学生的姓名与成绩，找出分数最高的那个并输出

运行结果

```
张三 92.5  李四 78.0
王五 85.5
最高：张三 92.5
```

要点

best 记的是下标，不是分数本身

找到人以后，姓名成绩一起取出来

例 3 · 按成绩排序

```
struct Student { char name[20]; double score; };
void sort(struct Student a[], int n) {
    int i, j; struct Student tmp;
    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - 1 - i; j++)
            if (a[j].score < a[j + 1].score) {
                tmp = a[j];  // 整份搬走
                a[j] = a[j + 1];
                a[j + 1] = tmp;
            }
}
// main 里读入 3 人后调用 sort(a, 3) 再遍历输出
```

题目

把 3 个学生按成绩从高到低排序，排序后完整输出每个人

运行结果

```
排序前：张三 李四 王五
第 1 名 张三 92.5
第 2 名 王五 85.5
第 3 名 李四 78.0
```

要点

tmp 也是结构体，交换时整份搬

名字跟着成绩走，绝不会串味

例 4 · 平均分与高于平均


```c
struct Student { char name[20]; double score; };
int main(void) {
    struct Student a[5]; int i, cnt = 0;
    double sum = 0, avg;
    for (i = 0; i < 5; i++)
        scanf("%s%lf", a[i].name, &a[i].score);
    for (i = 0; i < 5; i++) sum = sum + a[i].score;
    avg = sum / 5;
    for (i = 0; i < 5; i++)
        if (a[i].score > avg) cnt++;
    printf("avg=%.1f cnt=%d", avg, cnt);
    return 0;
}
```

题目

读入 5 个学生的姓名与成绩，算全班平均分，并统计高于平均分的人数

运行结果

```
92.5 78.0 85.5 61.0 70.0
总分 387.0，平均 77.4
高于平均：3 人
```

要点

要两轮循环：先算完平均才能数人数

sum 用 double，写成 int 会丢小数

例 5 · 输出学生生日

```c
// 内层结构体：生日
struct Date { int year, month, day; };
struct Student {
    char name[20]; struct Date birthday;
};
int main(void) {
    struct Student s = {"张三", {2006, 3, 15}};
    printf("%s 的生日：", s.name);
    printf("%d-%d-%d", s.birthday.year,
           s.birthday.month, s.birthday.day);
    return 0;
}
```

题目

学生信息里带一个生日结构体，输出这个学生的姓名和生日

运行结果

```
张三 的生日：2006-3-15
年：2006
月：3    日：15
```

要点

两层点：先取 birthday 再取 year

初始化时内层也要用一对花括号

常见错误盘点

六个高频坑

哈尔滨工程大学

✗ 定义结尾忘分号：结构体定义最后那个分号漏掉了

花括号结束必须写 }; 少一个分号，下一行立刻报错

✗ 点与箭头混用：拿着指针写 p.age ，拿着变量写 s->age

看左边是什么：变量用点，指针用箭头

✗ 结构体名与变量名混淆：拿类型名去访问成员

struct Student 是类型， s 才是变量，成员只能从变量取

✗ scanf 读成员忘加取地址符：写成 scanf("%d", s.age)

除了数组名，成员前面一律要加 & 符号

✗ 传值修改不带回：函数里改了形参，实参一动不动

想改实参就把形参写成指针，调用时传地址进去

✗ typedef 顺序写反：把别名写到了花括号前面

别名夹在右花括号和分号中间，写成 } Student;

小结与预告

结构体把不同类型捆成一个整体，一个学生终于是一个变量。

变量用点取成员，指针用箭头， typedef 让类型名短一点。

结构体数组装下一整班，排序时交换的是整个学生，名字不会串味。

下节预告

文件：把数据存到硬盘上

今天我们花力气录入的学生信息，程序一关就全没了。学完文件，就能把它写到硬盘上，关机也不丢。

下次再打开程序，一整班学生的名字和成绩还能原样读回来。

哈尔滨工程大学